

Multiclass Anomaly Analysis in Microservice Architectures Using Distributed Trace Data

Shane Gannon

Computing Department, Atlantic Technological University (ATU)

Email: L00196954@atu.ie

Abstract—The FIRM microservice tracing dataset contains over 56 million requests across four microservice applications. The dataset contains clear and measurable latency degradation patterns, particularly in tail-latency behaviour and scenario-level interference effects. The analysis aims to extract system level insights from large scale distributed data. A distributed analytics pipeline was implemented using Databricks and PySpark to assess if big data analytics can support real world cloud debugging through modelling and pattern matching. The findings are structured to address the main analytical questions: how latency-degradation appears across the four applications, how interference affects service performance, and how effectively the selected models detect degraded requests. The results demonstrate that the dataset contains measurable and clear degradation patterns. Degraded services can be identified with strong performance and localise across applications. Random Forest can provide slightly better predictions but at the expense of compute power and timing - the faster binary logistic regression is very capable. It shows that Random Forest is capable of learning effectively from the original latency values without requiring baseline-normalised inputs.

Index Terms—Predictive analytics, modelling, dataset, evaluation, visualisation

I. INTRODUCTION

Microservice architectures have become widely adopted in cloud and distributed computing [1]. Microservice applications are built on multiple independent components that communicate through APIs to deliver business capabilities. Maintaining and supporting microservices is complex because they run independently in dynamic environments, where execution paths can vary and performance data is distributed across many services. Traditional manual debugging and issue triage are not practical in these systems. Enterprise applications can leverage big data analytics to manage the masses of data generated by cloud applications [2] .

The scope of this analysis examines microservice latency figures across 4 microservice applications. Latency is defined as the duration of each service's span for a request, capturing both the time spent processing the request within the service and the communication delays between services. The study addresses typical big data challenges [3] – extracting system level insights from large scale distributed tracing data. The goal of this exercise is to investigate the FIRM microservice tracing dataset containing over 56 million requests across four applications. and answer the following

- Can large-scale microservice tracing data be processed using a distributed analytics pipeline?

- How does service interference impact request latency?
- Can machine learning models be used to predict degraded requests/degraded services?

To support the investigations distributed analytics pipeline was implemented using Databricks and PySpark to assess if big data analytics can support real world cloud debugging through modeling and pattern matching. The results provide faster incident triage, reduced incident resolution time, better service level support through early warning systems as services and applications begin to slow down and eventually fail.

II. DATASET DESCRIPTION

The dataset that was used for the investigations was the FIRM distributed tracing dataset taken from the Illinois Data Bank [4] . The dataset was created to support analysis of performance issues in microservice systems with anomalies injected to enable controlled exploratory analysis [5] . The original dataset consists of 72 CSV files which were processed using Databricks.

Once imported, the data set consists of 56.88 million records of latency information relating to the 4 microservice based applications. Each record represents a single request trace through one of the four microservice applications. While the data contains fields to support debugging such as the ingest file name there is a core set of data that is used for data processing and analysis.

- trace_id: A unique identifier for one request
- service_latency_fields: Service latency time values e.g. l_uniqueid
- app: Application identifier
- scenario: label indicating the service where the latency degradation occurred
- label_binary: binary degradation label - 0 for non-degraded, 1 for degraded
- total_latency: total request latency derived from the trace

Social-network is the largest dataset with 25280000 traces, over 4.5 times greater than the smallest data set hotel-reservation. Each application contains a different number of services which result in different counts of latency features per application. This implies the complexity associated with the microservices, social network is again the largest of the application with 31 associated services. As shown in table XX the dataset has a strong imbalance of degraded requests versus non-degraded requests ranging from 86% in hotel-reservation

Application	Rows	Degraded Req	Non Degraded Req	Latency Features
hotel-reservation	5530000	4740000	790000	6
media-service	15800000	15010000	790000	19
social-network	25280000	24490000	790000	31
ticket-booking	10270000	9480000	790000	12

Fig. 1. Application data summary

to 97% in social-network. This has implications for model evaluation, making metrics such as F1 score, ROC-AUC, and PR-AUC [6] more informative than accuracy alone.

Fig. 1 provides a high level summary of the dataset, including the application names, total row count degraded and non-degraded counts and the number of latency features or services associated with the application.

III. METHODOLOGY

This section provides an overview of the methodology employed to clean, prepare and analyse the dataset. I look into how the data is ingested, transformed and modelled to support analysis. This activity was structured through a layered approach [7]. The initial Bronze layer was used to ingest and store the raw data, Silver prepared the data for analysis and the Gold layer prepared the data for modelling. The exploratory nature of big data analysis provided many challenges resulting in reassessments changes in direction but all supported the analytic process. To achieve this I used Databricks, PySpark and Python with supporting libraries NumPy, Matplotlib Spark MLlib and SparkML.

A. Data Preparation

The FIRM dataset consisted of 72 CSV files containing raw tracing data. Due to the size of the dataset, initial attempts to upload the files to Databricks volume failed. So the Databricks CLI was used to transfer the files to cloud storage. Once uploaded PySpark was used to convert the raw CSV files into Parquet format for efficient storage and processing. Once the data was loaded the process of preparing the data for analysis could commence. Cleaning and data validation tests were conducted to identify if there were any null latency values which could cause processing issues in the pipelines. Data validity was also assessed by testing trace_id values for duplicates. In all cases, the dataset emerged without issue. Extreme values in all applications were retained as high latency values singled out service degradation and formed the core of the analysis. Rather than skewing the data and representing noise to be removed this data was very necessary to forming conclusions. A key step in the data preparation was the creation of request-level latency data sets, allowing analysis to be performed at the application and scenario level. The datasets were also restructured to support trace identification by application scenario e.g. “search” in the case of hotel reservation. A binary feature was created to indicate trace degradation status called is_degraded to identify a trace as degraded or non-degraded. The datasets remained separated by application throughout most of the pipeline because each

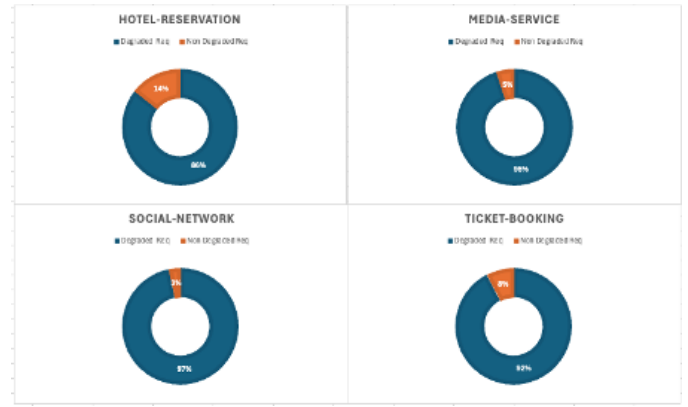


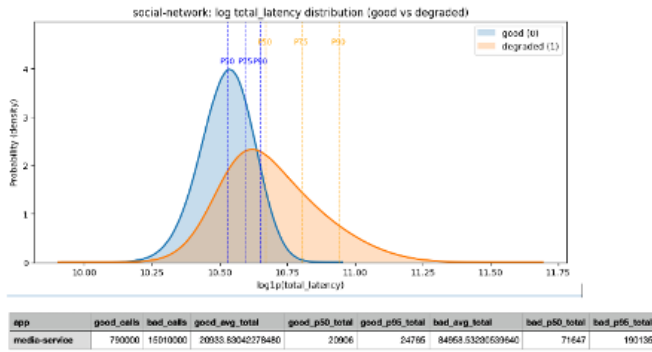
Fig. 2. degraded vs non degraded per Application

application contained a different number of service latency features associated with each application. The datasets were merged when cross application analysis was required. Initial designs considered creating a single dataset for all applications with zero filled unsupported service fields. However due to data size limitations, and the additional level data processing complexity meant that the application structure needed to be retained.

Exploratory data analysis was carried out on the Silver Datasets before data modelling took place. This stage was used to validate the prepared dataset and assess if they were suitable for downstream feature engineering and modelling. The analysis highlighted issues and valuable insights that supported further investigations. Scenario-level summaries were created by grouping traces according to their interference labels and calculating summary latency statistics for each group. This allowed for comparison of scenarios and proved out that there were real scenarios to test and noteworthy differentiation between applications, services and traces that supported the thesis and further investigation. Class balance is a datapoint which shows the breakdown of degraded and non-degraded traces within the dataset. EDA highlighted that the data was heavily skewed towards degraded data. This highlighted that model scores would need to take the skewed nature of the data into account when assessing models and performance.

Latency distributions were examined and plotted to assess the spread of latency values, the proportion of fast, slow and the presence of extremes. This confirmed there were patterns in degraded and non-degraded and these patterns could be analysed. The data also informed where the value was in the data. The long tails at P95 for degraded services contrasted with the short tails of the more uniform non-degraded data. This highlighted the need to retain all data as there was value in the edges.

In summary this stage of the pipeline provided the foundation of the investigations to come. It required a level of exploratory and iterative development. The processing was visited a number of times as data size, data quality and learnings resulted in rethinking how the data should be structured. The efforts at this stage of development ensured consistency across



the dataset and supported reliable EDA. The EDA activity confirmed there were questions and insights in the dataset and it shaped the analysis and tools to be used in the downstream modelling .

B. Feature Engineering

Feature engineering was carried out in the Gold layer to transform the cleaned Silver datasets into model-ready inputs for binary, multiclass, and cross-application predictive modelling. This stage produced two output data sets that were used in the downstream modelling tasks. The two outputs of this stage were Gold_raw and Gold_norm, representing raw latency and normalised latency figures. Both datasets contain the core request level variables prepared in the Silver layer including:

- Application level identifiers
- Degradation identifiers
- Scenario identifiers
- Total latency

The gold_raw dataset retained individual microservice latency figures

- latency figures for each Service

The gold_norm dataset replaces service latency figures in favour baseline-relative engineered features

- Delta figures for each service
- Ratio figures for each service

Delta figures: service latency minus the corresponding average service latency under the no-interference baseline.

Ratio figures: service latency divided by the corresponding average service latency under the no-interference baseline.

This was done because raw latency figures are not directly comparable across applications and scenarios, having a baseline relative features could provide models with more meaningful insights independent of applications.

Generic scaling such as z-score standardisation was not applied. Instead latency values were normalised relative to the no-interference baseline as raw values were not comparable across applications and scenarios. The normalised feature set offer the best opportunity to show how abnormal a given trace or service may be. This was specially relevant for the degradation detection as this captures deviations from normal behaviour.

- Goal 1: degraded vs non-degraded prediction
- Goal 2: identify which service was degraded
- Goal 3: compare gold_raw and gold_norm for Random Forest
- Goal 4: train on three applications and test on a held-out fourth application using gold_raw-derived features to evaluate cross-application generalisation.

Note dimensionality reduction and aggregation was not required for this dataset as they were structured appropriately for the next stage of modelling.

C. Model Development

This subsection outlines the predictive models used in the study and explains how each model was designed to address a specific analytical question. The modelling process moved from binary degradation detection, to multi-class degraded-service prediction, to non-linear model comparison, and finally to cross-application evaluation.

1) *Goal 1 – Binary classification:* [8] . The question – can a request be classified as degraded vs non-degraded using latency-based features was addressed using a binary logistic regression as the baseline classifier. This provides a method to model probability of a binary response using multiple predictors [8] . This model supported the use case as it was interpretable. – coefficients provided evidence of the influence of services on degraded predictions. The model also scaled well. Databricks was able to handle the processing and provided results in a short time. The model used gold_raw data set to provide the predictors. As with all model inputs in this investigation the dataset was divided into a 80/20 train test split with a seed = 42 to ensure the split is repeatable. The regression was trained in Spark ML – sparks built-in distributed machine learning tools. Tuning was limited as the baseline performed strongly, however tests were run to increase max iterations from 30 to 50 which did not deliver any meaningful improvements. Overall, Logistic Regression provided an interpretable and computationally efficient baseline for comparison with the more complex models used in later goals.

2) *Goal 2 – Multinomial logistic regression:* [8] Goal 2 asks can the model identify which service was degraded rather than simply detect if degradation occurred. To support this Multinomial Logistic Regression was used as it learns the probabilities of several possible classes, then chooses the most likely one. The model was fed the only degraded traces from the gold_raw data set. Training and validation was again carried out on a per application basis. Initial investigations started with a Random Forest multiclass model, and they performed strongly for some of the models. Issues were encountered with Social-network as processing ran into memory issues in the Databricks environment being used. In an effort to manage memory numTrees, maxDepth and maxBins were adjusted with some minor successes however a full run on all applications was not achievable. This resulted in a switch to multinomial logistic regression which avoided

the memory issues. The model proved successful and provided useful results.

3) *Goal 3 – Raw vs Normalised feature representation comparison:* Goal 3 asked does a non-linear model improve degraded trace detection. It also tested whether normalised engineered features (gold_norm) improved results compared with raw latency features (gold_raw). In this case a binary Random forest model [8] was used. Despite the issues in goal 2. The binary nature of the model and a restructured driver worker code pattern allowed the model to complete successfully. As memory issues impacted the Random Forest Multiclass model the code was separated to worker and driver notebooks allowing memory to free up after execution of an application. This model took both the gold_raw and gold_norm for the datasets for comparison. For tuning Random Forest hyper-parameters were set to modest values to reduce usage and improve execution. trees, depth, and bins were reduced to make the model feasible in Databricks – RF_TREES = 50, RF_MAX_DEPTH = 10, RF_MAX_BINS = 32 It should also be noted that execution took much longer than linear methods.

4) *Goal 4 – Cross application evaluation.* : Goal 4 addressed the question can degradation patterns learned from some applications could generalise to support analysis for an unseen application. In this case binary logistic regression was chosen because it had already shown strong performance in goal 1 and provided a stable, interpretable and efficient model. Because the 4 application feature sets contained service latency columns gold_raw summary features were used so there was a consistent model to work across the applications. The models were trained 4 times each time leaving one application out in order to carry out testing. Normalised fields:

- svc_nonzero_cnt: number of service latency columns in the trace that have a value other than 0.
- svc_max: The largest single service latency value in the trace.
- svc_sum: The sum of all service latency values in the trace.
- svc_mean: The average service latency value across the service columns in the trace
- total_latency_x: total latency value for the trace

These fields were created in the goal 4 notebooks as they are specific to this goal and not part of the base dataset. No tuning was required as this have been carried out from previous model tests.

In summary, the four modelling goals formed a coherent progression from detection, to localisation, to model and feature comparison, and finally to generalisation. This structure allowed the study to test not only whether latency-based degradation could be predicted, but also whether the same modelling approach remained effective across different applications.

D. Model Evaluation

In this section outlines the evaluation framework used to assess model performance. This will inform of the validity of the models used. This data will be carried forward to the results section also as model performance comparisons contribute to a large part of the overall analysis evaluation is not only used to measure predictive accuracy but also how reliable the predictions are. note: accuracy results comparisons are discussed in more detail in the results section

Metric	Goal 1	Goal 3	Better Model	Description
Accuracy	0.9626	0.9665	Goal 3	Overall proportion of correct predictions
Precision	0.9810	0.9857	Goal 3	How often degraded predictions were correct
Recall	0.9787	0.9781	Goal 1	How many degraded requests were identified
F1 score	0.9797	0.9818	Goal 3	Balance between precision and recall
ROC-AUC	0.9499	0.9543	Goal 3	Class separability across thresholds
PR-AUC	0.9973	0.9977	Goal 3	Precision-recall performance under imbalance

Fig. 3. Linear vs non linear model comparison

1) *Binary model evaluation – Goals 1 and 3:* Goal 1 used binary logistic Regression and goal 3 used binary random forest. Both notebooks used binary evaluators at the application level to assess the models and identify degraded traces. This provided an opportunity to compare linear and non-linear models. The following is a comparison table of the evaluation metrics used for the models. This included Accuracy, Precision, Recall, F1 score, ROC-AUC and PR-AUC. Confusion matrix counts were also created.

Goal 1 performed strongly as a binary baseline, achieving Accuracy of 0.9574–0.9686, F1 of 0.9770–0.9840, ROC-AUC of 0.8591–0.9804, and PR-AUC of 0.9948–0.9990 across the four applications, showing that gold_raw latency features were effective for degraded vs non-degraded prediction. Goal 3 performed strongly as a binary Random Forest model, achieving Accuracy of 0.9641–0.9688, F1 of 0.9803–0.9841, ROC-AUC of 0.8552–0.9915, and PR-AUC of 0.9942–0.9992 across the four applications using gold_raw, indicating slightly stronger overall binary performance than Goal 1, albeit at higher runtime. One important metric to consider for the Random Forest model is the compute time was calculated at 65 minutes to run the models for gold_raw and gold_norm

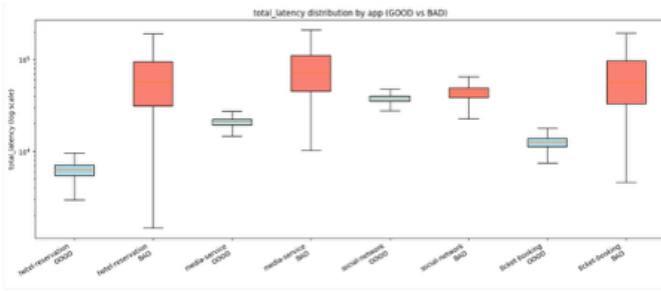
2) *Goal 2 model used Multinomial Logistic regression for degraded service prediction.* : This was a more complex modelling task than binary degradation detection and so required multiclass evaluation measures. Metrics used included Accuracy – providing a measure of how often the degraded service was predicated correctly, F1 score- a calculation of the balance between precision and recall.

3) *Goal 4 evaluated pooled applications with leave-one-out testing* : [9]. In terms of evaluation used the similar metrics , Accuracy, Precision, Recall, F1 score, however a pooled ROC-AUC was calculated in order to assess if a global model could separate degraded and non-degraded traces across all applications. Goal 4 achieved a pooled cross-application ROC-AUC of 0.9560, while leave-one-app-out F1 remained strong across all four applications (0.9745–0.9796), indicating that the degradation signal generalised well to unseen applications.

Overall, the evaluation framework was designed to match the project’s three main modelling challenges: • binary degradation detection • multi-class degraded-service localisation • cross-application generalisation and the chosen metrics allowed conclusions to be drawn about both predictive effec-

Goal	Model	Main Metrics Used	What the Metrics Tested	Key Takeaway
Goal 1	Logistic Regression	Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC	Binary degradation detection quality	Baseline model performed strongly across all apps
Goal 2	Multinomial Logistic Regression	Accuracy, F1, confusion matrix, per-class metrics	Service-level localisation quality	Harder task than Goal 1, focused on degraded service identification
Goal 3	Random Forest	Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC	Whether a nonlinear model and engineered features improved detection	Random Forest gave modest gains, especially on some apps
Goal 4	Logistic Regression	ROC-AUC, Accuracy, F1, Recall, Precision	Whether learned degradation behaviour transferred to unseen apps	Cross-app performance remained strong

Fig. 4. Goal to Model Summary



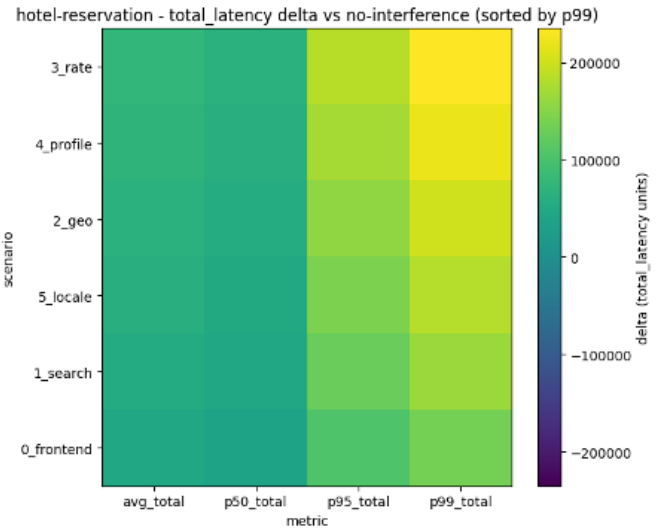
tiveness and practical deployment suitability.

IV. RESULTS AND ANALYSIS

This section presents the outcomes of the exploratory analysis and predictive modelling stages of the project. The findings are structured to address the main analytical questions: how latency degradation appears across the four applications, how interference affects service performance, and how effectively the selected models detect degraded requests, identify degraded services, and generalise across applications. A central question of this project asks can big data analysis support real world cloud triage. The section begins with key descriptive and latency-based findings from the exploratory analysis, before moving to the results of Goals 1 to 4 and the comparative performance of the modelling approaches.

1) *General data results.* : Initial EDA results showed that the project analysed 56.88 million traces across 4 applications. Social network was the largest of the application datasets with 25,288,000 rows, hotel reservation was the smallest with 5,530,000 rows. The feature sets in the data vary from 6 to 31. Latency analysis has provided some insight into the data set. Non-degraded traces showed tighter dispersion and shorter latency time than degraded traces. This is key to understanding the data set and identifying problems. Degraded traces presented worse tail latency behaviour with the strongest differences appearing in the p95 and p99 rather than averages.

The boxplot shows the latency distributions for Good vs Bad across all 4 applications. In all cases the bad distributions sit higher on the latency scale than the good distributions demonstrating that bad traces are consistently slower, median



App	Accuracy	Precision	Recall	f1	Roc_auc	Pr_auc
hotel-reservation	0.966810863837102	0.9963618674078610	0.9648033663900756	0.980328855366000	0.9800377141950300	0.9970021774093870
media-service	0.9573550731399780	0.97566329268592	0.9795424319052280	0.9775990141784960	0.9800619629718400	0.9989714027655480
social-network	0.9685527800699740	0.9688213246172480	0.999711370668510	0.9840239857708810	0.8591450711913250	0.9947590978853060
ticket-booking	0.9577390609340040	0.9831671385470290	0.970848833839460	0.9769691580735280	0.9803802943745080	0.9984385991158920

Fig. 5. Goal1 logistic Regression Results

latency is always higher and the spread of traces is wider. This also supports the decision to keep extreme values as they have a major influence. Hotel-reservation P95 comparisons highlight the point with good = 8,391 and bad = 160,898. The hotel reservation heatmap supports this demonstrating that total latency changed relative to the no-interference baseline across hotel-reservation scenarios, strongest degradation occurred in the tail metrics (especially p95 and p99) for scenarios such as 3_rate, 4_profile, and 2_geo.

2) *Goal 1 results – binary classification.* : Goal 1 tested if degraded and non-degraded requests could be predicated with a binary logistic regression. Again this was done on a per application level. The model performed strongly on all applications. Accuracy and precision were above .95 on all applications. This is a good indication that models could be used to predict if there was a problem with a trace in the system.

The ability to identify degraded traces through Recall was also strong with social-network – the largest and most complicated dataset showing .99. Roc-AUC was weakest on social network at .85 indicating that the model's overall ability to separate degraded and non-degraded traces was weaker for social-network than for the other applications. PR-AUC was consistently very high across all four applications (0.9948–0.9990), indicating that the binary Logistic Regression model maintained strong precision–recall performance despite the class imbalance. PR-AUC was important because it measured degraded-class performance more fairly than accuracy in a highly imbalanced dataset.

Goal 2 – Degraded service localisation Goal 2 moved deeper into the dataset by attempting to identify which ser-

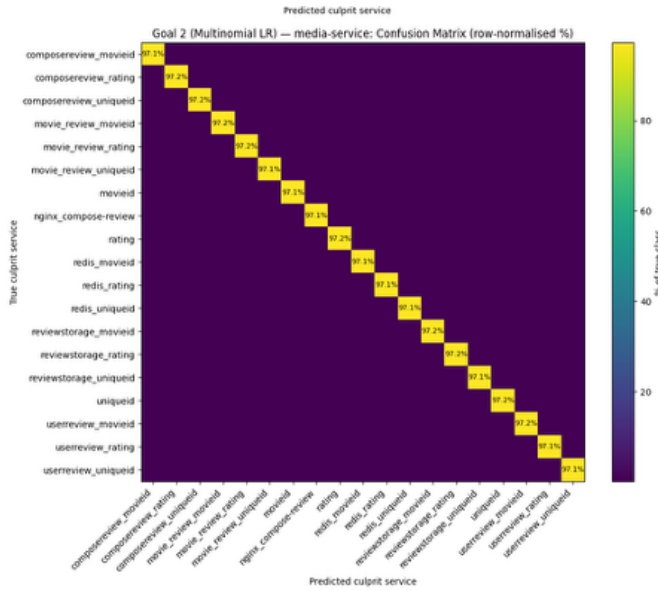


Fig. 6. Media Service level heat-map

App	RF (Random Forest) Accuracy	RF Precision	Logistic regression LR Accuracy	LR Precision	RF vs LR F1	RF precision vs LR
hotel-reservation	0.969996944	0.995797	0.966811	0.996362	0.00018608	0.028986478
media-service	0.96305915	0.985261	0.957355	0.975663	0.008950842	0.027906051
social-network	0.968757051	0.968757	0.968553	0.968821	0.000204271	0.000204271
ticket-booking	0.964065473	0.993081	0.957739	0.983167	0.006326412	0.035342439

Fig. 7. Non linear vs linear model figures

vice was degraded. Despite being a more difficult prediction task than binary degradation detection, the results remained strong. This was especially true for hotel-reservation, media-service, and ticket-booking, which returned Accuracy and F1 scores between 0.9714 and 0.9727. Performance was lower in social-network, which returned an Accuracy and F1 score of 0.8941. This is not surprising, as social-network was the most complex application, containing 31 service classes and the largest degraded dataset. Confusion matrices were created for all applications to demonstrate service-level performance. The media-service row-normalised confusion matrix has been included as a representative example. The dominant diagonal and very limited off-diagonal spread indicate that the model was highly effective at distinguishing between degraded services, with per-class performance consistently around 97.1

3) *Goal 3:* Goal 3 tested a number of hypotheses. Asking if applying a nonlinear model improves prediction and if normalised data provide more accuracy than raw data 1 Do non-linear models improve binary prediction.

The comparison table shows that Random Forest outperformed logistic regression across the four applications although gains were modest. This suggests that Logistic Regression may be the more practical option for near-real-time latency debugging, as the performance gains from Random Forest were relatively small when compared with the additional runtime cost. It should also be noted that Random Forest computation took considerably longer than logistic regression demonstrated by the associated graph.

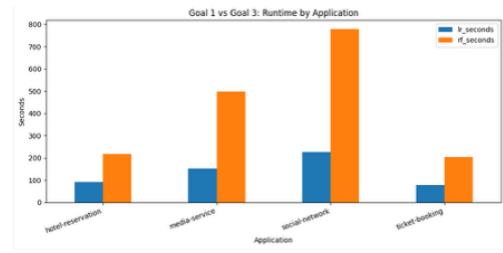


Fig. 8. Normalised vs Raw results

app	Normalised F1 Acc	Normalised Precision	Raw F1 Accuracy	Raw Precision
hotel-reservation	0.9682697857095760	0.9968475337838210	0.9669969441280380	0.9957973421204780
media-service	0.9608729664403550	0.9691600196697020	0.9663059146854730	0.9852611242677300
social-network	0.968757842130461	0.968757842130461	0.9687570508519560	0.9687570508519560
ticket-booking	0.9636799595794980	0.9908396806737360	0.9640654730352320	0.9930814997062520

This would point to logistic regression being more practical for real time application of latency debugging applications. Minor gains in accuracy at the expense of time computing cost and interpretability may not prove beneficial.

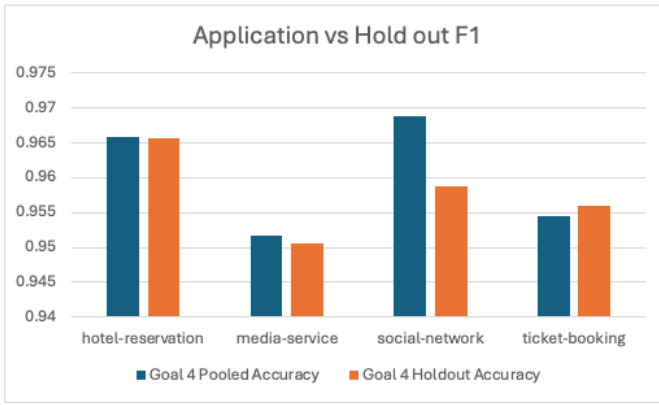
2 Do normalised datasets provide improved performance over raw data. The Random Forest model was run with normalised and raw data. The results were then compared for accuracy and are outlined in the table below.

Table X shows that the normalised feature representation did not outperform the raw feature representation in Goal 3. Raw features produced equal or better Accuracy and precision across all four applications, with the largest gap visible in media-service, suggesting that Random Forest was already able to learn effectively from the original latency values without requiring baseline-normalised inputs.

4) *Goal 4 :* Goal 4 tested if degradation patterns could transfer across application. This would enable new application upgrades or even new applications to enter the prediction system and provide meaningful results. The model was trained on 3 applications and tested on a holdout application, mimicking how the system would operate if a new or previously unseen application entered the system. These results were compared against goal 1 results to assess if the model would still provide accurate results. The table below and the associated bar chart show that performance remained close to the Goal 1 within-application baseline across all four applications. The largest drop was observed for social-network and media-service, but the reductions in both Accuracy and F1 were still relatively small. This suggests that the latency-based degradation signal was not fully application-specific and could generalise effectively to unseen systems.

Application	Goal 1 Accuracy	Goal 4 Holdout Accuracy	Goal 1 F1	Goal 4 Holdout F1
hotel-reservation	0.9668	0.9656	0.9803	0.9796
media-service	0.9574	0.9505	0.9776	0.9745
social-network	0.9686	0.9588	0.984	0.9789
ticket-booking	0.9577	0.9559	0.977	0.9758

Fig. 9. standard vs holdout accuracy comparison



Overall, the results show that the FIRM tracing dataset contains clear and measurable degradation patterns, particularly in tail-latency behaviour and scenario-level interference effects. The predictive modelling stages demonstrated that degraded requests could be identified with strong performance, that degraded-service localisation was feasible, and that a shared degradation signal could generalise across applications. In real world terms this answers that big data analyse can help identify anomalies in cloud applications. Goal 2 shows that not only can it identify degraded traces it can be more granular and help identify the actual affected service. Goal 3 attempts to identify the best models for near real time analysis. It shows that Random Forest can provide slightly better predictions but at the expense of compute power and timing – the faster binary logistic regression is very capable. Also normalised data requiring additional processing does not offer more value than Raw data. Finally goal 4 shows that additional services/applications can be added to the model without retraining and will provide usable predictive results. These findings address the core analytical questions of the project and provide the basis for the final conclusions, limitations, and recommendations presented in the next section.

V. CONCLUSIONS

The project leveraged the FIRM distributed data set to investigate microservice latency degradation in order to assess could big data analysis practices support distributed cloud based debugging and early warning systems. This can provide faster incident triage, earlier warning systems and more targeted debugging activities. Goal 1 asked could machine learning models support identification of degraded traces and even the origination services. This was confirmed by creating highly accurate model prediction models Goal 3 asked could nonlinear and normalised data improve prediction. The research supports this however gains were minimal. The final Goal confirmed that the models could take in new data and continue to provide accurate predictions. The project had some limitations, the dataset was heavily skewed towards degraded systems and the dataset was very structured compared to raw microservice latency logs. Future work should address these gaps to ensure real world value. The project succeeded in its

goal to create a distributed big data pipeline and create an test prediction models that are informative and accurate and can support engineering teams deliver quality robust microservice applications.

REFERENCES

All references must follow IEEE style and be cited numerically within the text.

REFERENCES

- [1] amazon, “Aws microservices,” 1 2026. [Online]. Available: <http://aws.amazon.com/microservices/>
- [2] L. Akmeemana, C. Attanayake, H. Faiz, and S. Wickramanayake, “Gal-mad: Towards explainable anomaly detection in microservice applications using graph attention networks,” 4 2025. [Online]. Available: <http://arxiv.org/abs/2504.00058>
- [3] IBM, “What is big data,” 1 2026. [Online]. Available: <https://www.ibm.com/think/topics/big-data>
- [4] Q. Haoran, B. S. S., J. Saurabh, K. Z. T., and I. R. K., “Pre-processed tracing data for popular microservice benchmarks,” 2020. [Online]. Available: <https://databank.illinois.edu/datasets/IDB-6738796>
- [5] M. Panahandeh, N. Ezzati-Jivan, A. Hamou-Lhadj, and J. Miller, “Efficient unsupervised latency culprit ranking in distributed traces with gnn and critical path analysis,” in *ICPE 2024 - Companion of the 15th ACM/SPEC International Conference on Performance Engineering*. Association for Computing Machinery, Inc, 5 2024, pp. 62–66.
- [6] A. Géron, *Hands-on Machine Learning With Scikit-Learn, Keras and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, 3rd ed. O’Reilly Media, Inc., 1 2023. [Online]. Available: <https://research.ebsco.com/linkprocessor/plink?id=c9ec11c7-5669-33a8-a448-99979a80ba09>
- [7] Databricks, “Medallion architecture,” 1 2026. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>
- [8] P. Bruce, A. Bruce, and P. Gedeck, *Practical Statistics for Data Scientists : 50+ Essential Concepts Using R and Python*, 2nd ed. O’Reilly Media, 1 2020. [Online]. Available: <https://research.ebsco.com/linkprocessor/plink?id=edd08dd3-2e48-3987-969c-bcfe9e3e9875>
- [9] J. Brownlee, “Loocv for evaluating machine learning algorithms,” 8 2020. [Online]. Available: <https://machinelearningmastery.com/loocv-for-evaluating-machine-learning-algorithms/>